



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



1.2 数据结构的抽象形式



- **类型**

一组值的集合。可分为原子类型和结构类型。

- **数据类型**

定义：一组性质相同的值的集合，以及定义于这个值集合上的一组操作的总称。

- **C语言中的数据类型**

char int float double void

字符型 整型 浮点型 双精度型 无值



不同类型的变量，其所能取的值的范围不同，所能进行的操作不同。

例如：整型 (int)

值的范围是：-32768 ~ 32767（16位）

操作是：+，-，*，/，%（双目运算符）

+，-（单目运算符）

<，>，<=，>=，==，!=（关系运算符）



1. 数据类型是什么？

数据类型是一组值的集合以及定义在这些值上的一组操作的统称。

它有两个核心作用：

- **约束取值**：规定了变量或表达式可以取哪些值（例如，`int` 类型不能存储小数或字母）。
- **规定操作**：规定了能对这些值执行哪些运算（例如，`int` 类型可以进行 `+`, `-`, `*`, `/` 等算术运算，但不能进行字符串拼接）。

2. 为什么需要数据类型？

- **内存管理**：编译器或解释器根据数据类型来分配适当大小的内存空间（例如，在C语言中，`int` 通常占4字节，`char` 占1字节）。
- **数据完整性**：防止程序执行无意义的操作（例如，避免将一个人的年龄与他的名字相加）。
- **确定操作含义**：确保操作符的行为是明确的（例如，`+` 对数字是加法，对字符串是连接）。



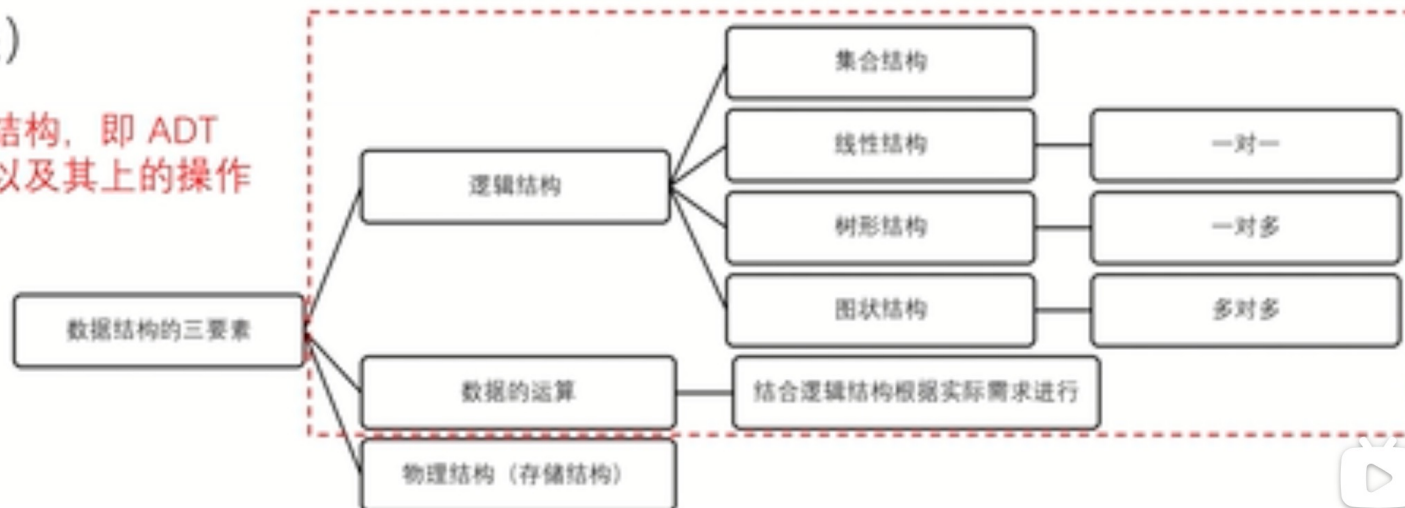
3. 分类与例子

- **基本/内置数据类型 (Primitive/Built-in Types):** 由编程语言直接提供。
 - 整数 (int): 如 42, -7
 - 浮点数 (float/double): 如 3.14, -0.001
 - 字符 (char): 如 'A', '9'
 - 布尔值 (boolean): true, false
- **复合/派生数据类型 (Composite/Derived Types):** 由基本类型或其他复合类型构造而来。
 - 数组 (Array): 相同类型元素的集合。
 - 结构体 (Struct) 或 记录 (Record): 不同类型元素的集合。
 - 字符串 (String): 通常是字符的序列（在很多语言中被视为基本类型，但本质上是复合的）。
 - 指针 (Pointer): 存储内存地址的类型。



- **数据类型**是一个值的集合和定义在此集合上的一组操作的总称
 - 原子类型：其值不可再分的数据类型
 - 结构类型：其值可以再分解为若干成分（分量）的数据类型
- **抽象数据类型**（Abstract Data Type, ADT）是抽象数据组织及与之相关的操作（与实现无关）

定义一种数据结构，即 ADT
包含逻辑结构以及其上的操作





In computer science, **an abstract data type (ADT)** is **a mathematical model for data types**, where a data type is defined by its behavior (semantics) **from the point of view of a user of the data**, specifically in terms of possible values, possible operations on data of this type, and the behavior of these operations.

*Liskov, Barbara; Zilles, Stephen (1974).
"Programming with abstract data types".
Proceedings of the ACM SIGPLAN Symposium
on Very High Level Languages. SIGPLAN
Notices. 9. pp. 50–59.*



Barbara Liskov



1.抽象数据类型是什么？

- **抽象数据类型 (ADT)** 是**数学模型**，它仅从逻辑上定义了一个数据对象、其数据元素之间的关系以及可对其执行的操作。它强调的是“**做什么**”，而不是“**怎么做**”。
- 它的核心思想是**封装**和**信息隐藏**。用户只需要知道它能完成什么功能，而无需关心其内部具体是如何实现的。

2. 为什么需要ADT？

- **封装复杂性**：将数据结构和算法的复杂实现细节隐藏起来，为用户提供一个清晰、简单的接口。
- **提高可维护性**：只要接口不变，内部实现的修改不会影响使用它的程序。
- **促进代码复用**：定义良好的ADT可以在多个项目中重复使用。
- **模型现实世界**：能够更自然地描述和模拟现实世界中的实体和问题。

3. 核心组成

一个ADT通常通过三个部分来定义：

- **数据对象**：描述了数据元素之间的逻辑关系。
- **数据关系**：描述数据元素之间的结构（如线性关系、树状关系等）。
- **操作**：一组可对数据执行的功能，这是用户与ADT交互的唯一途径。



4. 经典例子

- **栈 (Stack):** 后进先出 (LIFO) 的线性结构。
 - 操作: push (入栈), pop (出栈), peek (查看栈顶), isEmpty (判断是否为空)。
 - 不关心: 它是用数组实现的还是用链表实现的。
- **队列 (Queue):** 先进先出 (FIFO) 的线性结构。
 - 操作: enqueue (入队), dequeue (出队), isEmpty。
- **列表 (List):** 元素的有序集合。
 - 操作: insert, remove, get, size。
- **字典/映射 (Dictionary/Map):** 键值对 (Key-Value) 的集合。
 - 操作: put (添加/更新), get (根据键查找值), remove。
- **树 (Tree)、图 (Graph)** 等也都是ADT。



抽象数据类型 (*ADTs: Abstract Data Types*)

- 由用户定义，用以表示应用问题的数据模型
- 由基本的数据类型组成，并包括一组相关的服务（或称操作）
- 支持了逻辑设计和物理实现的分离，支持封装和信息隐蔽

抽象：抽取反映问题本质的东西，忽略非本质的细节



抽象数据类型的两种视图：

- **设计者的角度：**

根据问题来定义抽象数据类型所包含的信息，给出其相关功能的实现，并提供公共界面的接口。

- **用户的角度：**

使用公共界面的接口对抽象数据类型进行操作，不需要考虑其物理实现。对于外部用户来说，抽象数据类型应该是一个黑盒子。



自然数的抽象数据类型定义

ADT *NaturalNumber* is

/*objects: 一个整数的有序子集合, 它开始于0,
结束于机器能表示的最大整数(*MaxInt*)。*/

Function: 对于所有的 $x, y \in \textit{NaturalNumber}$;
 $\textit{False}, \textit{True} \in \textit{Boolean}$, $+$ 、 $-$ 、 $<$ 、 $==$ 、 $=$ 等都是可用的服务。

Zero() : NaturalNumber

返回自然数0



IsZero(x) :

Boolean

if $(x == 0)$ 返回 *True*

else 返回 *False*

Add(x, y) :

NaturalNumber

if $(x + y \leq \text{MaxInt})$ 返回 $x + y$

else 返回 *MaxInt*

Subtract(x, y) :

NaturalNumber

if $(x < y)$ 返回 0

else 返回 $x - y$

Equal(x, y) :

Boolean

if $(x == y)$ 返回 *True*

else 返回 *False*

Successor(x) :

NaturalNumber

if $(x == \text{MaxInt})$ 返回 x

else 返回 $x + 1$

end NaturalNumber



关系与联系

- **ADT**是数据类型的抽象和扩展。可以将基本数据类型看作是最底层的ADT。
- **ADT**需要通过具体的数据类型来实现。例如，Stack 这个ADT可以用 数组 或 链表 这两种具体的数据结构来实现。
- 编程语言中的“类” (**Class**) 是实现ADT的一种完美机制。类中的公有方法 (**Public Methods**) 就是ADT的操作接口，而类的私有字段 (**Private Fields**) 则隐藏了ADT的内部实现细节（即使用了何种具体的数据类型和算法）。

一个简单的类比

- 数据类型就像是一块砖、一片瓦、一根木头。你知道它们的大小、材质和基本用途。
- 抽象数据类型就像是一个房子的设计蓝图。蓝图定义了房子要有卧室、厨房、卫生间（操作），并规定了它们之间的关系（数据关系），但它没有规定墙体是用砖头还是用木头砌成的（实现细节）。
- 而用砖头、木头等材料根据蓝图建成真正的房子，这个过程就是实现ADT，建成的房子就是一个数据结构 (**Data Structure**)。



特性	数据类型 (Data Type)	抽象数据类型 (Abstract Data Type)
核心焦点	是什么 (值的集合和操作)	做什么 (逻辑行为和接口)
抽象级别	较低, 更具体	较高, 更抽象
实现细节	通常是已知的、透明的	完全隐藏, 只暴露接口
关注点	内存表示、取值范围	功能、行为、数据之间的关系
例子	int, float, char, array	Stack, Queue, List, Map



1.3 作为ADT的C++类



- **面向对象的概念**

面向对象 = 对象 + 类 + 继承 + 消息通信

面向对象方法中类的定义充分体现了抽象数据类型的思想（属性、操作）



C/C++基础

1. 什么是分支、循环？（if/else、for、while）
2. 什么是数组？
3. 什么是函数？
4. 什么是指针、什么是地址？
5. 什么是 struct 结构体？



// if-else 示例

```
int a = 10;
if (a > 5) {
    printf("a is greater than 5\n");
} else {
    printf("a is not greater than 5\n");
}
```

// else if 示例

```
int b = 20;
if (b > 30) {
    printf("b is greater than 30\n");
} else if (b > 20) {
    printf("b is between 20 and 30\n");
} else {
    printf("b is not greater than 20\n");
}
```

// switch 示例

```
int c = 3;
switch (c) {
    case 1:
        printf("c is 1\n");
        break;
    case 2:
        printf("c is 2\n");
        break;
    case 3:
        printf("c is 3\n");
        break;
    default:
        printf("c is not 1, 2, or 3\n");
        break;
}
```



1.声明和初始化数组:

// for 循环示例

```
for (int i = 0; i < 5; ++i) {  
    printf("i: %d\n", i);  
}
```

```
int arr1[10]; // 声明一个有10个整数的数组, 未初始化  
int arr2[5] = {1, 2, 3, 4, 5}; // 声明并初始化一个有5个整数的数组  
int arr3[5] = {1, 2}; // 剩余元素默认初始化为0  
char arr4[3] = {'a', 'b', 'c'}; // 字符数组初始化
```

// while 循环示例

```
int j = 0;  
while (j < 5) {  
    printf("j: %d\n", j);  
    j++;  
}
```

2.访问数组元素:

```
int n = arr2[1]; // 访问第二个元素, 值为2
```

3.遍历数组:

```
for (int i = 0; i < 5; i++) {  
    printf("%d ", arr2[i]);  
}  
// 输出: 1 2 3 4 5
```

// do...while 循环示例

```
int k = 0;  
do {  
    printf("k: %d\n", k);  
    k++;  
} while (k < 5);
```

4.修改数组元素:

```
arr2[1] = 10; // 将第二个元素的值修改为10
```

5.计算数组长度:

```
int length = sizeof(arr2) / sizeof(arr2[0]); // 计算数组长度
```




1. 函数概述

作用1: 将一段经常使用的代码封装起来, 减少重复的代码

作用2: 一个较大的程序, 一般分为若干个程序块, 每个模块实现特定的功能

2. 函数的定义

5个步骤

- i. 返回值类型 (一个函数可以返回一个值, 在函数中定义)
- ii. 函数名 (给函数取一个名字)
- iii. 参数列表 (使用该函数时, 传入的数据)
- iv. 函数体语句 (函数内需要执行的语句)
- v. return表达式 (和返回值类型挂钩, 函数执行完后, 返回相应的数据)

语法:

返回值类型 函数名 (参数列表)

{

函数体语句

return表达式

3. 函数的调用

语法: 函数名 (参数)

*函数定义里小括号内称为形参, 函数调用时传入的参数称为实参



在C/C++中，指针是一个变量，其值为另一个变量的地址。

1.定义和使用指针

```
int x = 10;
int *pointer;
pointer = &x; // 将变量x的地址赋值给指针pointer
```

2.通过指针访问变量

```
printf("%d\n", *pointer); // 输出10
```

3.指针和数组

```
int arr[5] = {1, 2, 3, 4, 5};
int *p;
p = arr;
for (int i = 0; i < 5; i++) {
    printf("%d ", *(p + i)); // 输出数组的元素
}
```

4.指针和字符串

```
char *str = "Hello, World!";
while (*str != '\0') {
    printf("%c", *str); // 输出字符串的每个字符
    str++;
}
```



在C/C++中，**struct**是一种构造类型，它可以将不同的数据类型组合在一起，形成一个新的数据类型。

解决方案1：定义和使用**struct**

```
struct Student {
    char name[20];
    int age;
    float score;
};

int main() {
    struct Student stu; // 声明变量
    strcpy(stu.name, "Tom"); // 给name赋值
    stu.age = 18; // 给age赋值
    stu.score = 92.5; // 给score赋值

    printf("Name: %s\n", stu.name);
    printf("Age: %d\n", stu.age);
    printf("Score: %.1f\n", stu.score);

    return 0;
}
```

解决方案2：定义和使用匿名**struct**

```
struct {
    char name[20];
    int age;
    float score;
} stu; // 声明匿名结构体类型的同时声明了变量stu

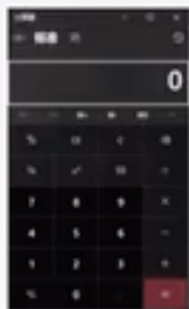
int main() {
    strcpy(stu.name, "Tom"); // 给name赋值
    stu.age = 18; // 给age赋值
    stu.score = 92.5; // 给score赋值

    printf("Name: %s\n", stu.name);
    printf("Age: %d\n", stu.age);
    printf("Score: %.1f\n", stu.score);

    return 0;
}
```



从面向过程到面向对象



编程实现计算器:

```
float add(float a, float b);
```

```
float sub(float a, float b);
```

```
float mul(float a, float b);
```

```
float div(float a, float b);
```

问题/场景/系统越来越复杂...



编程实现动物园模拟:

```
void AeatB(A a, B b);
```

```
void AeatC(A a, C c);
```

```
void BfightD(B b, D d);
```

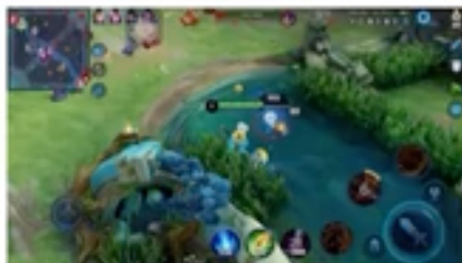
```
void Esleep(E e);
```

..... (很多很多过程)



实现一个商城系统:

用户登录、购买商品、退换货、
商品上下架、优惠券、库存、商家、
评论..... (很多很多过程)



做一个游戏:

有玩家、野怪、地图等实体
玩家又有血条、蓝条等属性
每个实体还能进行攻击、放技、行为.... (很多很多过程)





“面向过程”和“面向对象”

- 面向过程就是分析出解决问题所需要的步骤，用函数把这些步骤一步一步实现，使用的时候一个个依次调用就可以了；
- 面向对象是把构成问题事务分解成各个对象，建立对象的目的不是为了完成一个步骤，而是为了描述某个事物在整个解决问题的步骤中的行为。
- 可以拿生活中的实例来理解面向过程与面向对象，如五子棋，面向过程的设计思路就是首先分析问题的步骤：
 - 1、开始游戏；
 - 2、黑子先走；
 - 3、绘制画面；
 - 4、判断输赢；
 - 5、轮到白子；
 - 6、绘制画面；
 - 7、判断输赢；
 - 8、返回步骤2；
 - 9、输出最后结果。
- 把上面每个步骤用不同的方法来实现。



“面向过程”和“面向对象”

- 如果是面向对象的设计思想来解决问题，整个五子棋可以分为
 - 1、黑白双方，这两方的行为是一模一样的；
 - 2、棋盘系统，负责绘制画面；
 - 3、规则系统，负责判定诸如犯规、输赢等。
 - 第一类对象（玩家对象）负责接受用户输入，并告知第二类对象（棋盘对象）棋子布局的变化，棋盘对象接收到了棋子的变化就要负责在屏幕上面显示出这种变化，同时利用第三类对象（规则系统）来对棋局进行判定。
- 可以看出，面向对象是以功能来划分问题，而不是步骤。
- 同样是绘制棋局，这样的行为在面向过程的设计中分散在了多个步骤中，很可能会出现不同的绘制版本，因为通常设计人员会考虑到实际情况进行各种各样的简化。
- 而在面向对象的设计中，绘图只可能在棋盘对象中出现，从而保证了绘图的统一。



用C++语言描述面向对象程序

- C++的函数特征
- C++的数据声明
- C++的作用域
- C++的类
- C++的对象
- C++的输入/输出
- C++的函数
- C++的参数传递
- C++的函数名重载和操作符重载
- C++的动态存储分配
- 友元(friend)函数
- 内联(inline)函数
- 结构(struct)与类
- 联合(Union)与类



- 函数**声明** (原型)

- 函数**定义** (实现)

- 函数的**调用** (call)



函数的**参数**



函数体



函数的**返回值**





函数的目的

- 函数是面向过程的体现：输入(参数) -> 处理流程 -> 输出(返回值)
- 目的：将重复发生的过程进行统一描述，实现代码复用与解耦
 - **代码复用**：避免重复写相同代码
 - **解耦**：避免发生改动时牵一发而动全身





示例：实现一个「银行账户」抽象数据类型 (**BankAccount ADT**)

这个 BankAccount ADT 将：

- 1.隐藏其内部数据（账户号、余额），只通过公共接口进行交互。
- 2.提供一组明确的操作（存款、取款、查询等）。
- 3.封装业务规则（例如，取款不能导致余额为负）。

构造函数 (**Constructor**)

特征：函数名与类名完全相同，无返回类型。

作用：在对象创建时自动调用，用于初始化对象的状态（成员变量）。

重载：类可以有多个同名但参数列表不同的构造函数，编译器根据传入参数决定调用哪一个。

析构函数 (**Destructor**)

特征：函数名为 ~类名，无参数，无返回类型。

作用：在对象销毁时自动调用，用于清理资源（如释放内存、关闭文件等）。



常量成员函数 (Const Member Function)

特征：在函数声明末尾加上 `const` 关键字。

作用：承诺不修改调用它的对象的状态。`getBalance()` 和 `getAccountNumber()` 是典型的访问器 (**Accessor**) 函数，它们只查询而不修改数据，因此应声明为 `const`。

返回值类型 (Return Type)

特征：函数可以返回任何有效类型，包括 `void`（无返回值）。

作用：向调用者传递操作结果或信息。例如，`withdraw` 返回 `bool` 来指示操作成功与否，而 `deposit` 返回 `void`，只执行动作。

静态成员函数 (Static Member Function)

特征：使用 `static` 关键字声明。

作用：该函数与类的任何特定对象无关。它属于类本身，而不是类的实例。它不能访问普通的成员变量（因为它们属于对象），只能访问静态成员变量。



```
#include <iostream>
#include <string>

// 1. 使用 'class' 关键字来定义我们的抽象数据类型 (ADT)
class BankAccount {
// 2. 访问修饰符: private 部分封装 (隐藏) 了内部数据表示
private:
    std::string accountNumber; // 账户号
    double balance;           // 余额

// 3. 访问修饰符: public 部分定义了ADT的对外接口 (操作)
public:
    // 4. 【函数特征: 构造函数 (Constructor)】
    // - 函数名与类名完全相同: BankAccount
    // - 无返回类型 (连void都没有)
    // - 可以重载 (Overload) 。这里提供了两个构造函数, 特征不同: 参数列表不同。
    BankAccount(const std::string& accNum, double initialBalance)
        : accountNumber(accNum), balance(initialBalance) { // 成员初始化列表
        // 构造函数体: 可以添加验证逻辑
        if (initialBalance < 0) {
            balance = 0.0;
            std::cerr << "Warning: Initial balance cannot be negative. Set to 0.0." << std::endl;
        }
    }
}
```



// 重载的构造函数：只提供账户号，余额默认为0

```
BankAccount(const std::string& accNum)
    : accountNumber(accNum), balance(0.0) {}
```

// 5. 【函数特征：析构函数 (Destructor)】

// - 函数名为 ~类名: ~BankAccount

// - 无参数，无返回类型

// - 通常用于释放资源。本例中无动态内存，但显式声明是一个好习惯。

```
~BankAccount() {
    std::cout << "Account " << accountNumber << " is being closed." << std::endl;
}
```

// 6. 【函数特征：常量成员函数 (Const Member Function)】

// - const 关键字在函数参数列表之后

// - 表示该函数不会修改类的任何成员变量 (即对象状态)

// - 只能调用其他const成员函数

```
double getBalance() const {
    return balance;
}
```

```
const std::string& getAccountNumber() const {
    return accountNumber;
}
```



```
// 7. 【函数特征: 修改器成员函数 (Modifier Member Function)】
// - 函数会改变对象的内部状态 (balance)
void deposit(double amount) {
    if (amount > 0) {
        balance += amount;
        std::cout << "Deposited: $" << amount << std::endl;
    } else {
        std::cerr << "Deposit amount must be positive." << std::endl;
    }
}

// 8. 【函数特征: 返回值类型为 bool】
// - 该操作可能成功也可能失败, 通过返回值告知调用者结果
bool withdraw(double amount) {
    if (amount > 0 && balance >= amount) {
        balance -= amount;
        std::cout << "Withdrew: $" << amount << std::endl;
        return true; // 取款成功
    } else {
        std::cerr << "Withdrawal failed. Insufficient funds or invalid amount." << std::endl;
        return false; // 取款失败
    }
}

endl;
```




```
// 9. 【函数特征：静态成员函数 (Static Member Function)】
// - 使用 static 关键字声明
// - 不属于任何特定对象，而是属于整个类
// - 只能访问静态成员变量，不能访问普通成员变量 (如balance)
static void displayBankPolicy() {
    std::cout << "Bank Policy: Minimum balance requirement may apply." << std::endl;
}
}; // 注意：类定义以分号 ; 结束

// 主函数，演示如何使用 BankAccount ADT
int main() {
    // 10. 使用构造函数创建对象 (实例化ADT)
    BankAccount myAccount("SB-12345", 500.0); // 调用两个参数的构造函数
    BankAccount yourAccount("SB-67890");      // 调用一个参数的构造函数

    // 11. 通过公共接口 (.操作符) 与对象交互
    std::cout << "My account number: " << myAccount.getAccountNumber() << std::endl;
    std::cout << "Initial balance: $" << myAccount.getBalance() << std::endl;

    myAccount.deposit(200.0); // 修改状态
    std::cout << "Balance after deposit: $" << myAccount.getBalance() << std::endl;
```



```
bool success = myAccount.withdraw(100.0); // 使用返回值判断操作结果
if (success) {
    std::cout << "Withdrawal successful. New balance: $" << myAccount.getBalance() <<
std::endl;
}

success = myAccount.withdraw(1000.0); // 这个操作会失败
if (!success) {
    std::cout << "Could not complete withdrawal." << std::endl;
}

// 12. 调用静态成员函数，无需对象实例，通过类名调用
BankAccount::displayBankPolicy();

// 13. 析构函数会在main函数结束时自动调用（对象超出作用域）
return 0;
}
```

```
My account number: SB-12345
Initial balance: $500
Deposited: $200
Balance after deposit: $700
Withdrew: $100
Withdrawal successful. New balance: $600
Withdrawal failed. Insufficient funds or invalid amount.
Could not complete withdrawal.
Bank Policy: Minimum balance requirement may apply.
Account SB-67890 is being closed.
Account SB-12345 is being closed.
```




over: 重复

load: 装载

函数重载 overload

- 思考：函数解决了什么问题？
 - 避免重复书写相同的过程/流程代码，实现代码复用和解耦
- 函数还有什么问题？
 - C++是强类型语言，同一套操作若要用在**不同类型**/数量的参数上，则需要编写**不同函数**！
- 所以呢？
 - 调用方需针对不同的实参写不同的函数调用代码，if-else增多，需判断类型信息
 - 函数名称不同，有多少种参数列表就需要多少个函数名
 - ...
- 解决思路：允许多个**同名函数**存在，分别处理不同类型/数量的参数...
- 这就是**函数重载**

```
string myAdd(int a, int b) {  
    return std::to_string(a) + std::to_string(b);  
}  
  
string myAdd(int a, string s) {  
    return std::to_string(a) + s;  
}  
  
string myAdd(string s, int a) {  
    return s + std::to_string(a);  
}  
  
string myAdd(string s1, string s2) {  
    return s1 + s2;  
}
```

```
cout << myAdd(111, 999);  
cout << myAdd(111, "abc");  
cout << myAdd("abc", 111);  
cout << myAdd("abc", "def");
```

Function overloading

```
int max(int x, int y){ return x < y ? y : x;}  
float max(float x, float y) { return x < y ? y : x;}  
string max(string& x, string& y) { return x < y ? y : x;}
```

A template function

```
template <class T>  
T my_max(T x, T y) { return x < y ? y : x; }
```



内联(inline)函数

- 函数的目的：将重复发生的流程统一起来，实现代码复用和解耦
- 但是！如果某个函数的功能**非常简单**，又被**反复调用**的话，存在什么问题？
 - 提示：“函数调用”这个行为本身也是有一定开销的（调用栈）
- 那么调用这个函数的“额外开销”占比就很大了
- 此时可以**建议**编译器在编译时将函数直接在调用处**展开**，避免函数调用行为的额外开销
- 这就是**函数内联**

本想饮水机复用 → 结果走去茶水间的开销大于接一杯水的收益 → 不如每个办公室配一台饮水机

• 一家公司平面图：





virtual “虚”

- 回顾：继承来源于同一对象可能有多重身份，且这些身份有层级递进关系
- 实践中会有这类情况：
 - 父类中的某些行为需要在子类中被更加具体地细化
 - 父类中的某些行为不可确定，必须在子类中实现
- 于是**虚函数**的概念产生：
 - 父类的虚函数可以在子类中被**重写**(override)，即重新实现，但**参数和返回值必须保持一致**！
- 含有虚函数的类叫做虚类

```
class Human {  
public:  
    virtual void say() {  
        cout << "I'm human";  
    }  
};
```

func() 被 override

```
class Student : public Human{  
public:  
    void say() {  
        cout << "I'm a student";  
    }  
};
```